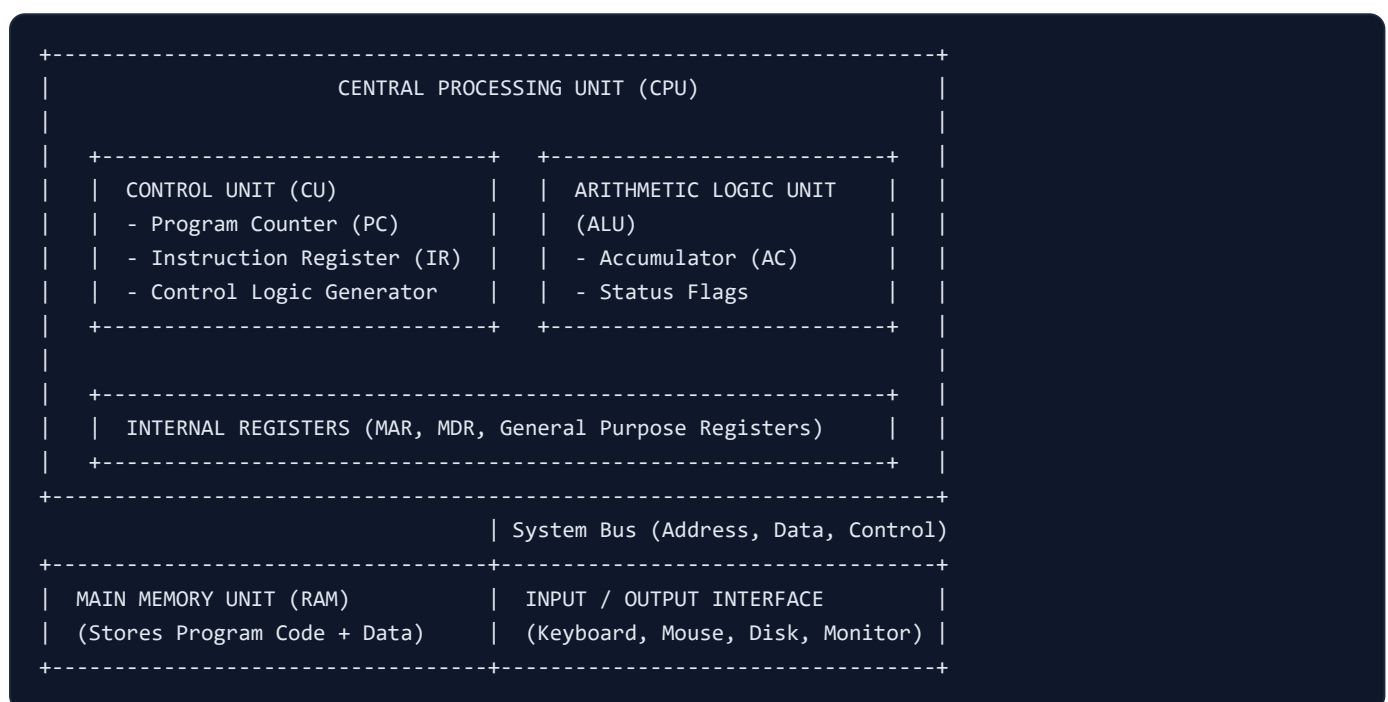


10-Mark Questions & Answers — MPCA Unit 1: Overview of Computer Architecture & Organization

Q1. Explain in detail the Von Neumann model of computer architecture. Also discuss the block-level description of functional units (CU, ALU, Memory, I/O) with a diagram.

1. Introduction to Von Neumann Architecture

Proposed by John von Neumann in 1945, the **Von Neumann Model** laid the foundation for modern general-purpose computing based on the **Stored-Program Concept**.



2. Detailed Functional Units

1. Control Unit (CU):

- **Program Counter (PC):** Holds the memory address of the next instruction to fetch.
- **Instruction Register (IR):** Holds the current instruction fetched from memory during decoding.
- **Instruction Decoder & Timing Logic:** Translates opcodes into control signals triggering functional execution.

2. Arithmetic Logic Unit (ALU):

- Executes arithmetic operations (**ADD** , **SUB** , **MUL**) and logical operations (**AND** , **OR** , **XOR** , shifts).
- Contains temporary registers such as the **Accumulator (AC)** and **Status Flags** (Zero, Carry, Sign, Overflow).

3. Main Memory Unit:

- Linear array of addressable storage locations containing both program code and working variables.
- Interfaced via **Memory Address Register (MAR)** and **Memory Data Register (MDR)**.

4. Input/Output (I/O) Interface:

- Manages communication between external peripheral devices and internal system registers using polling, interrupts, or Direct Memory Access (DMA).

Q2. Describe all the addressing modes used in the 8086 family with suitable examples for each. Explain how each mode calculates the effective address.

1. Effective Address (EA) Calculation Formula

In 8086, the **Physical Address (20-bit)** is calculated as:

Physical Address = (Segment Register × 16) + Effective Address (EA)
where EA = Base Register + Index Register + Displacement

2. Detailed 8086 Addressing Modes

Addressing Mode	Effective Address Formula	Example Instruction	EA & Physical Address Explanation
Immediate	No EA (Operand in code).	MOV AX, 5000H	Data 5000H is encoded directly inside instruction bytes.
Register Direct	No EA (Operand in register).	MOV AX, BX	Data taken directly from CPU register BX .
Direct Memory	EA = Displacement	MOV AX, [2000H]	EA = 2000H. PA = (DS × 16) + 2000H.
Register Indirect	EA = [BX/BP/SI/DI]	MOV AX, [BX]	If BX = 1500H , EA = 1500H.
Based	EA = Base + Disp	MOV AX, [BX + 08H]	If BX = 1000H , EA = 1000H + 08H = 1008H.
Indexed	EA = Index + Disp	MOV AX, [SI + 10H]	If SI = 2000H , EA = 2000H + 10H = 2010H.
Based-Indexed	EA = Base + Index	MOV AX, [BX + SI]	If BX = 1000H, SI = 0050H , EA = 1050H.
Based-Indexed with Disp	EA = Base + Index + Disp	MOV AX, [BX + SI + 04H]	If BX = 1000H, SI = 0050H , EA = 1054H.

Q3. Explain the evolution of computers from the first to the fifth generation, highlighting key technological changes and performance improvements at each stage.

+-----+
|
|COMPUTER GENERATIONS EVOLUTION|
|
|1st Gen (1940-56) : Vacuum Tubes ----> 1000s of Operations/sec|
|2nd Gen (1956-63) : Transistors -----> 100,000s Operations/sec|
|3rd Gen (1964-71) : Integrated Circuits (SSI/MSI) -> Millions Ops/sec|
|4th Gen (1971-Pres) : Microprocessors (VLSI/ULSI) -> Billions Ops/sec|
|5th Gen (Future) : Artificial Intelligence & Quantum Computing|
+-----+

1. 1st Generation (1940-1956): Vacuum Tubes

- Technology: Thermionic vacuum tubes for logic; magnetic drums for memory.
- Features: Enormous physical size, high heat generation, machine language programming.
- Example: ENIAC, UNIVAC-I, EDVAC.

2. 2nd Generation (1956-1963): Transistors

- Technology: Bipolar junction transistors replaced bulky vacuum tubes.
- Features: Smaller footprint, lower power consumption, Assembly & High-Level languages (FORTRAN, COBOL).
- Example: IBM 7094, CDC 1604.

3. 3rd Generation (1964-1971): Integrated Circuits (ICs)

- Technology: Small/Medium Scale Integration (SSI/MSI ICs) combining dozens of transistors on silicon chips.
- Features: Keyboards, monitors, Operating System software layer, timeshare processing.
- Example: IBM System/360, PDP-8.

4. 4th Generation (1971-Present): Very Large Scale Integration (VLSI/ULSI)

- Technology: Single-chip Microprocessors containing millions/billions of transistors.
- Features: Personal Computers (PCs), laptops, high-speed RAM, graphical user interfaces (GUI), Internet.
- Example: Intel 8086, Pentium, Core i9, Apple M3.

5. 5th Generation (Present & Future): AI & Parallel Supercomputing

- Technology: Ultra Large Scale Integration (ULSI), Quantum computing, Neural processing units (NPU)s.
- Features: Parallel Processing (MIMD), natural language processing, autonomous AI decision systems.

Q4. Discuss various performance metrics used to evaluate computer architecture with formulas, and explain Amdahl's Law with a numerical example.

1. Performance Metrics & Formulas

1. CPU Execution Time (T_CPU):

$$T_{\text{CPU}} = \text{Instruction Count (IC)} \times \text{CPI} \times \text{Clock Cycle Time (T)} = (\text{Instruction Count} \times \text{CPI}) / \text{Clock Rate (f)}$$

2. Cycles Per Instruction (CPI):

$$\text{CPI} = \text{Total CPU Clock Cycles} / \text{Instruction Count}$$

3. Millions of Instructions Per Second (MIPS):

$$\text{MIPS} = \text{Instruction Count} / (T_{\text{CPU}} \times 10^6) = \text{Clock Rate} / (\text{CPI} \times 10^6)$$

4. Floating-Point Operations Per Second (FLOPS):

$$\text{FLOPS} = \text{Total Floating-Point Operations} / \text{Execution Time in Seconds}$$

2. Amdahl's Law

Amdahl's Law quantifies the maximum expected overall system speedup when only a fraction **f** of a task is enhanced by a factor **s**:

$$\text{Speedup}_{\text{Overall}} = 1 / [(1 - f) + (f / s)]$$

3. Numerical Example

- **Problem Statement:** A program spends **80%** of its total execution time on vector matrix multiplication. An engineer adds a hardware matrix accelerator that speeds up vector matrix operations by **5 times (s = 5)**. Calculate

the overall speedup achieved.

- **Given Data:**

- Fraction enhanced (**f**) = **0.80**
- Un-enhanced fraction (**1 - f**) = **0.20**
- Enhancement speedup (**s**) = **5**

- **Calculation:**

$$\text{Speedup_Overall} = 1 / [(1 - 0.80) + (0.80 / 5)] = 1 / [0.20 + 0.16] = 1 / 0.36 = 2.78x \text{ Speedup}$$

- **Result:** The overall system execution speed increases by **2.78 times** (or a 178% speedup).